Exercícios

Exercício 1 - Conceitos Base de Containers e Imagens

Criar um servidor nginx com base na imagem existente no dockerhub.

Este container deverá estar disponível no porto 8081.

Validar que o servidor está à escuta no porto 8081.

Questões:

1.1 Qual é o nome do container?

- Pode ser consultado via docker container ls
- O nome será diferente de estudante para estudante.
  - *If you do not assign a container name with the --**name** option, then the daemon generates a random string name for you. Defining a **name** can be a handy way to add meaning to a container. If you specify a **name**, you can use it when referencing the container within a Docker network. This works for both background and foreground Docker containers.* https://docs.docker.com/engine/reference/run/#name---name
  - Nomes por defeito são constituídos por "<adjectivo>_<nome cientistafamoso/hacker>" . Exemplo: strange_shaw
  - Nota: Não é possível ter o nome aleatório "boring_wozniak": https://github.com/moby/moby/blob/master/pkg/namesgenerator/names-generator.go

```
if name == "boring_wozniak" /* Steve Wozniak is not boring */ {
        goto begin
}
```

1.2 Renomear o container para webhost.

1.3 Remover o container.

1.4. Para recuperar o espaço perdido em images, containers, network, etc, pode ser feito o comando "docker system prune"

Exercício 2 – Conceitos Base de Containers e Imagens

Criar 5 servidores nginx à escuta do porto 8081 ao porto 8085.

Deverá ser usada o mesmo terminal e não recorrer a 5 terminais para criar os containers.

Questões:

1.1 Validar que os 5 containers estão a correr.

1.2 Quantas imagens existem associadas ao container?

1.3 Qual o consumo de cpu por cada container?

Exercício 3 – Comandos Exec e Bind mount

Criar um container nginx disponível no porto 8081 com o nome nginx. Deverá ser alterada a página default que aparece no endereço http://localhost:8081/.

Existem 2 formas de efetuar esta operação em containers. Ambas deverão ser experimentadas.

Método 1 – Entrar no container via bash e alterar o ficheiro default que aparece no endereço acima mencionado:

/usr/share/nginx/html/index.html

Questões:

- Parando e arrancando o container o que acontece à alteração efetuada?
- Apagando e criando novamente o container o que acontece à alteração efetuada?

Método 2: Alterar o ficheiro via bind mouting

1- Criar um índex.html no host:

```
2- <!doctype html>
3- <html lang="en">
4- <head>
5-   <meta charset="utf-8">
6-
7-   <title>Your 2nd Dockerfile worked!</title>
8-
9- </head>
10-
11- <body>
12-   <h1>You just successfully ran a container with a bind mount!</h1>
13-   <img src="https://media.tenor.com/images/a00314fbfd7cad10e508e6a69436bf74/tenor.gif" />
14- </body>
15- </html>
```

2- Associar esse ficheiro **via bind** ao container.

3- Validar que a nova página aparece corretamente.

Questões:

- Parando e arrancando o container o que acontece à alteração efetuada?
- Apagando e criando novamente o container o que acontece à alteração efetuada?
- docker container inspect retorna o bind efectuado?

Exercício 4 – Gestão de múltiplos Containers

Neste exercício iremos efectuar a gestão de múltiplos containers. Iremos correr um container com nginx, uma base de dados mysql e também um servidor apache httpd. Os três containers deverão ser corridos com –detach (ou –d) , e com o nome mencionado através do comando –name.

Nginx deverá estar à escuta no porto 80:80, httpd no porto 8080:80 e a base de dados mysql no porto 3306:3306.

A base de dados mysql permite criar uma password random. Para isso, quando se corre o container mysql, especificar a opção de ambiente com o –env (ou –e) passando o parâmetro MYSQL_RANDOM_ROOT_PASSWORD=yes

Posteriormente devem ser analisados os logs do container para obter a password.

No final apagar os 3 containers criados.

Nota1: Os comandos de stop e remove aceitam múltiplos containers Ids. Validar com o docker container ls que não existem containers a correr.

Nota2: docs.docker.com e o --help são sempre boas ajudas.

Exercício 5 – Docker DNS e Network

Validar como funciona o DNS. A partir do Docker Engine 1.11, podemos ter múltiplos containers numa network a responder ao mesmo endereço.

Neste exercício deve ser criada uma nova virtual network (a default é a bridge).

Deverão ser criados 2 containers com base na imagem elasticsearch:2.

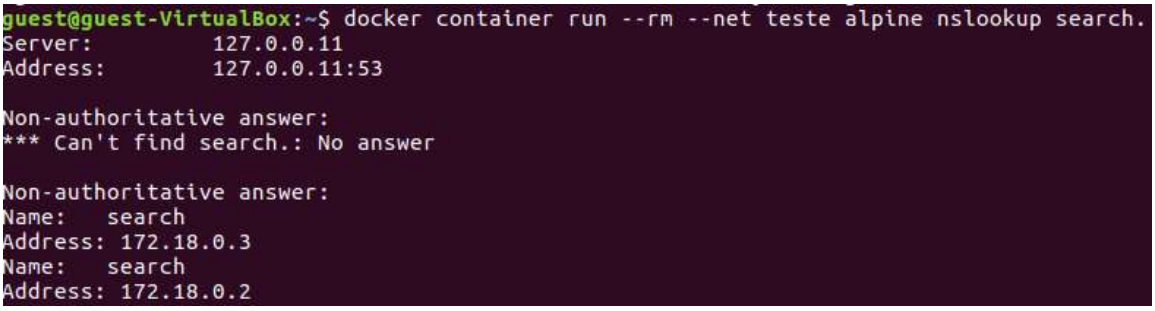
Usar -alias para dar um novo nome no DNS a ambos os containers. (https://docs.docker.com/engine/reference/commandline/network_connect/)

O resultado do comando docker container ls deverá ser similar ao seguinte (ter em atenção aos portos iguais):

```
guest@guest-VirtualBox:~$ docker container ls
CONTAINER ID   IMAGE             COMMAND                  CREATED              STATUS              PORTS               NAMES
ed54c758491e   elasticsearch:2   "/docker-entrypoint.…"   About a minute ago   Up About a minute   9200/tcp, 9300/tcp   admiring_goldwasser
fdae2afdb26a   elasticsearch:2   "/docker-entrypoint.…"   About a minute ago   Up About a minute   9200/tcp, 9300/tcp   bold_gagarin
```

Usar um container com base no alpine com o comando nslookup search e a –net anterior .

Deverá ser vísivel 2 containers como os ilustrados abaixo:



Usar o container centos com o comando curl -s search:9200 .

Deverá ser retornado as 2 instâncias de BD após algumas tentativas.

Exercício 6 – Migração de base de dados - Volumes

Deve ser criado um container postgres com a versão 9.6.1 e com um volume psql que mapeie no diretório usado para a DATA da BD de postgres.

Deve ser validado com recurso aos logs que a BD subiu corretamente.

Posteriormente deve ser parado o container.

Deve ser criado um novo container com base na versão 9.6.2 e com o volume anteriormente criado (psql). Validar que a BD subiu corretamente e que a versão da BD passou de 9.6.1 para 9.6.2.

Este é um caso prático em que os volumes fazem sentido, embora normalmente a migração de base de dados só seja possível em patchs.

Exercício 7 – Criação de imagens

Neste exercício vamos retornar a fazer o exercício 3, mas desta vez criando uma nova imagem com base no ficheiro referido no método 2 do exercício 3.

Deverá ser enviada essa imagem para o dockerhub.

Exercício 8 – Criação de novas imagens e push para o Docker Hub

8.1 Criar um servidor tomcat.

Pesquisar por um servidor tomcat no dockerhub (https://hub.docker.com/search?type=image)

Nota: repositórios começados com "_" são de images oficias suportadas. Exemplo: https://registry.hub.docker.com/_/ubuntu/

Caso não tenham, trata-se de versões suportadas pela comunidade.

Para instalação e alguma possível configuração, usar as instruções disponíveis na página da imagem (exemplo: https://hub.docker.com/_/tomcat)

8.2 Por defeito um servidor tomcat não possui o gestor ativo.

Ativar o gestor, tendo como referência o indicado aqui: https://octopus.com/blog/deployable-tomcat-docker-containers

8.3 Efetuar uma nova imagem que possua os 2 ficheiros acima mencionados.

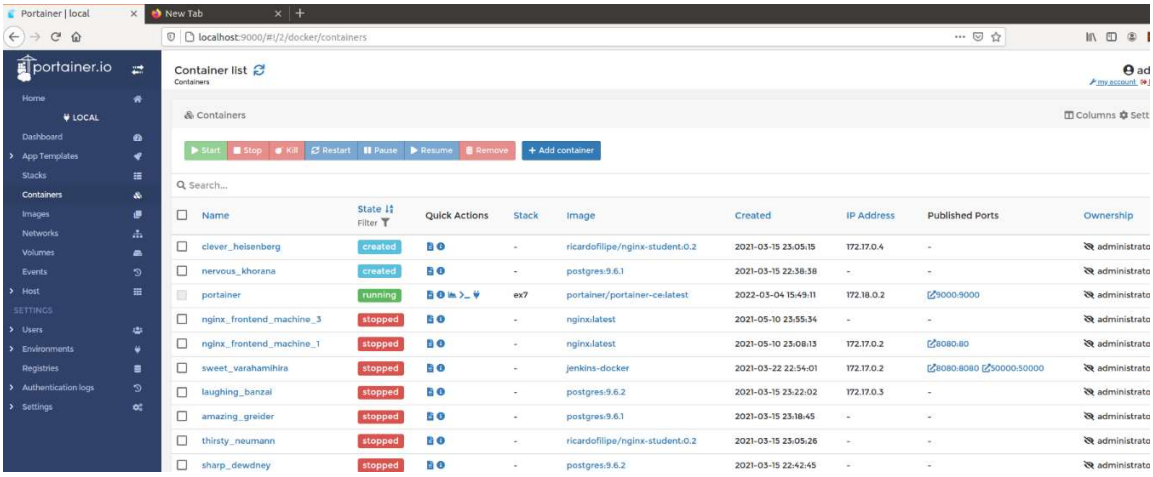
Essa imagem deverá ser carregada para o dockerhub.

Exercício 9 – Criar Jenkins via Docker Container

Iniciar um servidor Jenkins.
Deve ser consultada a página no dockerhub para validar como criar um container com base na imagem oficial.
docker run -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts

Exercício 10 – UI para gestão de containers e docker-compose

Vamos agora utilizar um UI para fazer a gestão dos nossos containers (quer seja em docker, swarm, ou Kubernetes). Para isso vamos usar o Portainer CE que permite fazer a gestão dos nossos containers (além de fazer monitoria), como se verifica pela imagem abaixo:



Nota: Pode ser consultado o site como exemplo de instalação usando o docker-compose.

Devem ser invocadas operações via Portainer para validar o correto funcionamento.

Exercício 11 – Criação de imagens mais complexas

11.1 Deverá ser criado um container com base na aplicação web para análise de dados apache zeppelin. Para isso pode usar-se a imagem existente no dockerhub.

11.2 Posteriormente, é pedido que essa mesma imagem tenha um jupyter notebook instalado, para assim a imagem conter 2 aplicações para análise de dados.

11.3 É também requisito que as seguintes bibliotecas python estejam disponíveis para utilização na imagem : xgboost , pandas , sklearn , imblearn , seaborn , numpy , matplotlib

11.4 Deverá ser possível correr o zeppelin , jupyter ou um comando mudando somente o parâmetro de entrada.

Exemplo:

#inicia a aplicação zeppelin

$docker container run -it -p 8080:8080 jupyter zeppelin

#inicia a aplicação jupyter

$docker container run -it -p 8080:8080 jupyter jupyter

#executa um comando dentro do container

$docker container run -it -p 8080:8080 jupyter echo “retorna esta string”

Posteriormente pode ser feito o build da imagem:

docker build -f Dockerfile --no-cache

11.5 Deverá ser utlizado um bind mount para que uma pasta do host esteja acessível dentro do container.

Deverá ser utlizado um bind mount fora do container para uso futuro.